

问题 2：NEP 势函数计算的 OpenMP 加速

2026 年 5 月 23 日

1 问题要求

问题 2 要求对分子动力学模块中的势函数计算进行 OpenMP 多线程加速。具体工作包括：

1. 分析势函数计算中的并行机会和数据依赖关系；
2. 实现 OpenMP 并行循环；
3. 正确处理线程私有数据和归约操作；
4. 测量不同线程数下的加速比；
5. 验证并行计算结果与串行计算结果一致。

本实现主要聚焦于 `source/source_md/potential/ml/nep` 目录下的 NEP CPU 势函数实现。

2 计算热点与数据依赖分析

NEP 势函数计算主要包括三个阶段：

1. 近邻表构建；
2. 描述符与神经网络能量导数计算；
3. 径向项、角向项和 ZBL 项的力累加。

其中，近邻表构建和描述符计算阶段已经包含了按原子划分的 OpenMP 并行循环。进一步分析后发现，`force path` 是仍需重点优化的计算热点。该部分主要包含如下形式的循环：

`for  $n_1 \in [0, N)$ , for each neighbor  $n_2$ .`

每一个原子对贡献都会同时更新两个原子的受力：

$$F_{n_1} += f_{12}, \quad F_{n_2} -= f_{12}.$$

同时，`virial` 数组也会按照原子索引进行更新。因此，如果直接使用

`#pragma omp parallel for`

对外层循环进行并行化，不同线程可能会同时写入同一个原子的 `force` 或 `virial` 分量，从而产生数据竞争。这会导致并行结果不稳定，甚至出现不可重复的数值误差。

3 实现方法

本次实现对以下 NEP force kernel 进行了并行化处理：

- `find_force_radial_small_box`
- `find_force_angular_small_box`
- `find_force_ZBL_small_box`

具体实现采用线程私有累加缓冲区：

1. 每个 OpenMP 线程分配私有 `force` 数组；
2. 每个 OpenMP 线程分配私有 `virial` 数组；
3. ZBL 路径额外使用私有势能数组；
4. 外层原子循环使用 `#pragma omp for` 进行并行划分；
5. 循环结束后，再将各线程的私有缓冲区归约到共享输出数组中。

这种设计避免了在最内层近邻循环中频繁使用 `atomic` 操作，也保持了原始力计算公式不变。其代价是额外的内存开销：

$$O(TN)$$

其中， T 表示 OpenMP 线程数， N 表示原子数。对于本次课程作业而言，这种方法实现清晰、线程安全，并且便于验证正确性。

4 正确性测试

本文增加了一个单元测试风格的测试程序，位置为：

`source/source_md/test/nep_omp_test.cpp`

测试使用仓库中已有的 Hf/O NEP 模型：

`tests/PP_ORB/nep_hfo2.txt`

该测试程序首先使用单线程计算一个确定性的 Hf/O 构型，并将其作为参考结果；随后使用四线程对同一构型再次计算，并逐分量比较势能、力和 `virial`。测试采用的误差容忍范围为：

$$|\Delta E| \leq 10^{-12}, \quad |\Delta F| \leq 10^{-10}, \quad |\Delta V| \leq 10^{-10}.$$

通过该测试，可以验证 OpenMP 并行化后是否保持了与单线程结果的一致性。

5 基准测试脚本

为了进一步测试不同体系规模 and 不同线程数下的性能表现, 本文增加了完整的 benchmark 脚本:

```
source/source_md/tools/run_problem2_nep_sweep.sh
```

该脚本会使用 OpenMP 编译独立 benchmark 程序, 并对不同体系规模和线程数进行扫描测试. benchmark 源文件为:

```
source/source_md/tools/nep_omp_sweep.cpp
```

复现实验可以使用如下命令:

```
cd source/source_md
tools/run_problem2_nep_sweep.sh ../tests/PP_ORB/nep_hfo2.txt \
64,216,512 \
1,2,4 \
10 \
reports/problem2_nep_sweep_results.csv
```

6 基准测试结果

本次 benchmark 使用 g++-15 编译, 编译选项为 -O2 和 -fopenmp. 计时过程中不重复统计模型文件加载时间; 每个 benchmark case 中, NEP 模型只初始化一次, 实际测量循环调用的是 NEP::compute.

原子数	线程数	平均时间 (ms)	加速比	最大 ΔE	最大 ΔF	最大 ΔV
64	1	3.2284	1.0000	0	0	0
64	2	1.7484	1.8465	0	8.60536×10^{-16}	1.77636×10^{-14}
64	4	1.0887	2.9654	0	8.32667×10^{-16}	2.13163×10^{-14}
216	1	10.0371	1.0000	0	0	0
216	2	5.1304	1.9564	0	1.16067×10^{-15}	1.77636×10^{-14}
216	4	2.8277	3.5496	0	1.40103×10^{-15}	2.13163×10^{-14}
512	1	28.7085	1.0000	0	0	0
512	2	15.4138	1.8625	0	1.61503×10^{-15}	2.13163×10^{-14}
512	4	9.7524	2.9437	0	1.49360×10^{-15}	2.13163×10^{-14}

表 1: 不同体系规模和线程数下的 NEP OpenMP 测试结果

7 结果分析

正确性测试结果表明, 在所有测试情形下, 势能与单线程参考结果完全一致. 力和 virial 的差异分别处于 10^{-15} 到 10^{-14} 的量级, 这与浮点数在不同累加顺序下产生的正常舍入误差一致.

因此，可以认为并行化后的 NEP force path 在数值上保持了正确性。

从性能结果看，加速效果较为明显：

- 对于 64 原子体系，四线程获得约 $2.97\times$ 加速；
- 对于 216 原子体系，四线程获得约 $3.55\times$ 加速；
- 对于 512 原子体系，四线程获得约 $2.94\times$ 加速。

因此，本实现基本满足问题 2 的主要要求：在 NEP 势函数计算热点中加入了 OpenMP 并行机制，使用线程私有缓冲区和归约处理共享输出依赖，并在多个线程数和多个体系规模下验证了串行与并行结果的一致性。

8 局限性与后续工作

当前实现使用了与原子数成正比的线程私有数组。该方法简单、安全，便于验证正确性，但其内存开销会随着线程数增加而增长。后续可以考虑使用原子分块、区域分解，或者在编译器支持充分的情况下使用 OpenMP 数组归约，以进一步降低内存开销。

此外，本次测试主要在最多四线程的本地环境下完成。未来如果能够在多核服务器上测试更大规模体系，可以进一步获得更完整的并行可扩展性曲线。

9 小结

本文针对 NEP 势函数计算中的 force path 进行了 OpenMP 并行优化。由于每个近邻原子对会同时更新两个原子的力和 virial，直接并行化会产生数据竞争。为此，本文采用线程私有 force、virial 和 ZBL 势能缓冲区，并在并行区域结束后进行统一归约，从而实现了线程安全的并行计算。

测试结果表明，并行版本在势能、力和 virial 上均与单线程结果保持一致，误差仅处于浮点舍入量级。同时，在 64、216 和 512 原子体系上，四线程均取得了接近三倍或超过三倍的加速效果。由此说明，本次优化有效完成了问题 2 所要求的 OpenMP 加速、数据依赖处理、性能测试和正确性验证。